

PS Lounge SaaS v4 — Security Hardening Implementation

جدول المحتويات

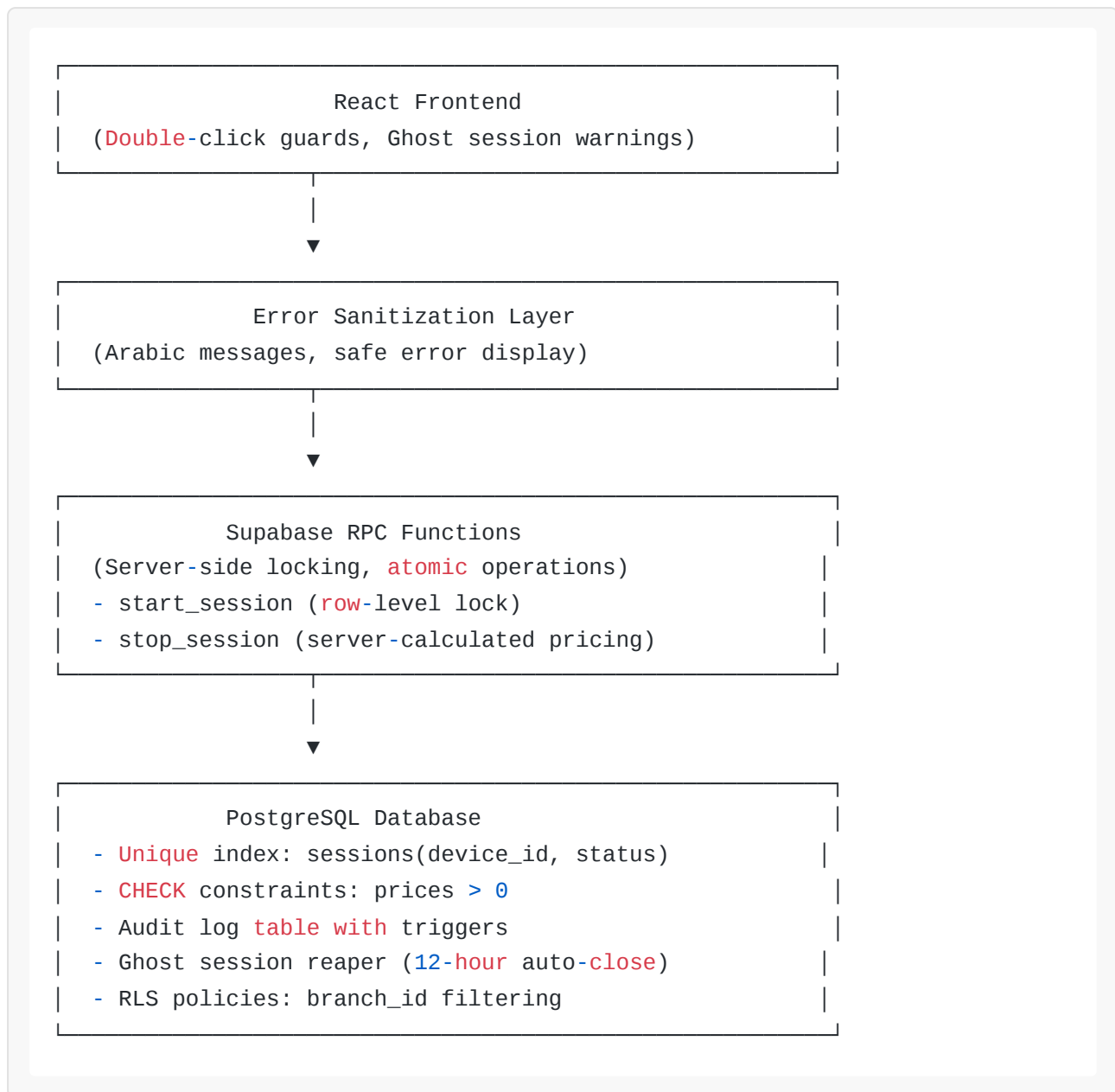
1. نظرة عامة على الهندسة المعمارية
 2. الميزات الأمنية المضافة
 3. معالجة الأخطاء والرسائل العربية
 4. حماية من الأخطاء المالية
 5. Supabase تعليمات النشر على
 6. اختبارات الوحدة
-

1 نظرة عامة على الهندسة المعمارية

المشكلة الأصلية

- **تكرار الجلسات:** يمكن لموظفين متعددين بدء جلسة على نفس الجهاز في نفس الوقت
- **التلاعب بالأسعار:** العميل يمكنه تعديل السعر قبل الإرسال لل خادم
- لا تصفي البيانات حسب الفرع Realtime **تسرب البيانات:** الاشتراكات
- **الجلسات المهجورة:** الجلسات الطويلة تبقى مفتوحة إلى الأبد
- **الأخطاء الخام:** رسائل الخطأ تفضح تفاصيل قاعدة البيانات

الحل المطبق



2 الميزات الأمنية المضافة

أ) منع تكرار الجلسات النشطة

الملف: drizzle/migrations/001_security_hardening.sql

```

-- Unique index prevents duplicate active sessions per device
CREATE UNIQUE INDEX idx_sessions_device_active
ON sessions(device_id, status)
WHERE status = 'active';

-- Row-level locking in start_session function
CREATE OR REPLACE FUNCTION start_session(...)
RETURNS TABLE(...) AS $$
BEGIN
    -- Lock the device row to prevent concurrent starts
    SELECT * FROM devices WHERE id = p_device_id FOR UPDATE;

    -- Check if session already exists
    IF EXISTS (SELECT 1 FROM sessions
               WHERE device_id = p_device_id
               AND status = 'active') THEN
        RAISE EXCEPTION 'DUPLICATE_SESSION';
    END IF;

    -- Insert new session
    INSERT INTO sessions(...) VALUES(...);
END;
$$ LANGUAGE plpgsql;

```

ب) حماية من التلاعب بالأسعار

الملف: src/lib/sessions.ts

```
// ❌ WRONG: Client sends price to server
export async function stopSession_OLD(sessionId: string, price: number) {
  return supabase
    .from('sessions')
    .update({ price_paid: price }) // ← Client can modify!
    .eq('id', sessionId);
}

// ✅ CORRECT: Server calculates price
export async function stopSession(sessionId: string) {
  const { data, error } = await supabase.rpc('stop_session', {
    p_session_id: sessionId,
  });
  // Server function calculates: (minutes / 60) * hourly_rate
}
```

الملف: drizzle/migrations/001_security_hardening.sql

```

-- Server-side pricing calculation
CREATE OR REPLACE FUNCTION stop_session(p_session_id UUID)
RETURNS TABLE(...) AS $$
DECLARE
    v_duration_seconds INTEGER;
    v_hourly_rate DECIMAL;
    v_price_paid DECIMAL;
BEGIN
    -- Get session and lock it
    SELECT * INTO v_session FROM sessions
    WHERE id = p_session_id FOR UPDATE;

    -- Calculate duration server-side (never trust client)
    v_duration_seconds := EXTRACT(EPOCH FROM (NOW() - v_session.started_at));

    -- Get locked hourly rate from device
    SELECT hourly_rate INTO v_hourly_rate FROM devices
    WHERE id = v_session.device_id;

    -- Calculate price: round up minutes
    v_price_paid := CEIL(v_duration_seconds / 60.0) / 60.0 * v_hourly_rate;

    -- Update session with calculated values
    UPDATE sessions SET
        ended_at = NOW(),
        duration_mins = CEIL(v_duration_seconds / 60.0),
        price_paid = v_price_paid,
        status = 'completed'
    WHERE id = p_session_id;

    RETURN QUERY SELECT * FROM sessions WHERE id = p_session_id;
END;
$$ LANGUAGE plpgsql;

```

Multi-tenancy (ج) تصفية البيانات حسب الفرع

الملف: src/lib/sessions.ts

```
// ADD branch_id filter to Realtime subscriptions
export function subscribeToSessions(callback: () => void, branchId?: string)
{
  return supabase
    .channel(channelName)
    .on(
      'postgres_changes',
      {
        event: '*',
        schema: 'public',
        table: 'sessions',
        // ✅ CRITICAL: Filter by branch_id to prevent cross-tenant leakage
        ...(branchId && { filter: `branch_id=eq.${branchId}` }),
      },
      callback
    )
    .subscribe();
}
```

الملف: src/hooks/useDevices.ts

```
export function useDevices(branchId?: string) {
  // Filter devices by branch_id
  let devQuery = supabase.from('devices').select('*');
  if (branchId) {
    devQuery = devQuery.eq('branch_id', branchId);
  }

  // Subscribe with branch_id filter
  const channel = subscribeToSessions(fetchAll, branchId);
}
```

د) اكتشاف الجلسات المهجورة

الملف: src/hooks/useDevices.ts

```
// Detect sessions at risk of being auto-closed
export function isGhostRisk(startTime: string): boolean {
  const hoursElapsed = (Date.now() - new Date(startTime).getTime()) /
3_600_000;
  return hoursElapsed > 6; // warn after 6 hours, auto-close at 12
}
```

الملف: src/components/devices/DeviceCard.tsx

```
// Visual warning for ghost sessions
{ghostRisk && (
  <div className="flex items-center gap-2 rounded-md bg-amber-500/20 border
border-amber-500/40 px-3 py-2">
    <span className="text-amber-600 dark:text-amber-400 text-sm">⚠️</span>
    <span className="text-amber-700 dark:text-amber-300 text-xs">
      جلسة طويلة - يرجى المراجعة
    </span>
  </div>
)}
```

الملف: drizzle/migrations/001_security_hardening.sql

```

-- Auto-close abandoned sessions after 12 hours
CREATE OR REPLACE FUNCTION close_ghost_sessions()
RETURNS TABLE(closed_count INTEGER) AS $$
BEGIN
    UPDATE sessions SET
        status = 'ghost_closed',
        ended_at = NOW(),
        notes = COALESCE(notes, '') || ' [Auto-closed after 12 hours]'
    WHERE status = 'active'
    AND started_at < NOW() - INTERVAL '12 hours';

    RETURN QUERY SELECT COUNT(*)::INTEGER FROM sessions
    WHERE status = 'ghost_closed'
    AND ended_at > NOW() - INTERVAL '1 hour';
END;
$$ LANGUAGE plpgsql;

-- Schedule the function to run every hour
SELECT cron.schedule('close-ghost-sessions', '0 * * * *',
    'SELECT close_ghost_sessions()');

```

هـ) حماية من النقرات المتكررة

الملف: src/components/devices/DeviceCard.tsx

```
const [isProcessing, setIsProcessing] = useState(false);

// Double-click guard: prevent concurrent operations
const handleEnd = async () => {
  if (!session || isProcessing) return; // ← Guard prevents duplicate clicks

  setIsProcessing(true);
  try {
    await stopSession(session.id);
    toast.success(`تمت الجلسة - ${estimatedPrice} جنيه`);
    onUpdate();
  } catch (error) {
    const appError = sanitizeError(error);
    toast.error(appError.message);
  } finally {
    setIsProcessing(false);
  }
};

// Disabled button while processing
<button
  onClick={handleEnd}
  disabled={isProcessing}
  className={clsx('btn-danger w-full py-2 text-sm',
    isProcessing && 'opacity-50 cursor-not-allowed')}
>
  {isProcessing ? 'إنهاء الجلسة' : '...إلتهاء...'}
</button>
```

3 معالجة الأخطاء والرسائل العربية

الملف: `src/lib/errors.ts`

```
// Custom error codes for our business logic
const DB_ERROR_CODES: Record<string, string> = {
  SESSION_NOT_FOUND: 'الجلسة غير موجودة أو تم إغلاقها بالفعل',
  DEVICE_UNAVAILABLE: 'الجهاز غير متاح حالياً',
  DUPLICATE_SESSION: 'يوجد جلسة نشطة بالفعل على هذا الجهاز',
};

// PostgreSQL error codes
const POSTGRES_ERROR_CODES: Record<string, string> = {
  '23505': 'يوجد تكرار في البيانات - يرجى المحاولة مرة أخرى',
  '23503': 'خطأ في البيانات المرتبطة',
  '23514': 'البيانات المدخلة تنتهك قواعد النظام',
  '40001': 'يرجى المحاولة مرة أخرى - تضارع في العمليات',
  '40P01': 'يرجى المحاولة مرة أخرى - تضارع في الوصول',
};

// Sanitize errors before showing to user
export function sanitizeError(error: unknown): AppError {
  // Log full error internally for debugging
  console.error('[PS Lounge Error]', error);

  // Check for our custom codes
  for (const [code, arabicMsg] of Object.entries(DB_ERROR_CODES)) {
    if (msg.includes(code)) {
      return { message: arabicMsg, code, isRetryable: false };
    }
  }

  // Check for PostgreSQL codes
  const pgCodeMatch = msg.match(/(\d{5})/);
  if (pgCodeMatch && POSTGRES_ERROR_CODES[pgCodeMatch[1]]) {
    const code = pgCodeMatch[1];
    return {
      message: POSTGRES_ERROR_CODES[code],
      code,
      isRetryable: ['40001', '40P01'].includes(code),
    };
  }

  // Safe fallback
```

```
return {  
  message: 'حدث خطأ غير متوقع. يرجى المحاولة مرة أخرى',  
  isRetryable: true,  
};  
}
```

4 حماية من الأخطاء العالية

CHECK Constraints

الملف: drizzle/migrations/001_security_hardening.sql

```
-- Prevent negative prices  
ALTER TABLE sessions ADD CONSTRAINT check_price_positive  
CHECK (price_paid >= 0);  
  
-- Prevent negative durations  
ALTER TABLE sessions ADD CONSTRAINT check_duration_positive  
CHECK (duration_mins > 0 OR duration_mins IS NULL);  
  
-- Prevent negative hourly rates  
ALTER TABLE devices ADD CONSTRAINT check_hourly_rate_positive  
CHECK (hourly_rate > 0);  
  
-- Prevent negative loyalty points  
ALTER TABLE sessions ADD CONSTRAINT check_loyalty_positive  
CHECK (loyalty_points >= 0 OR loyalty_points IS NULL);
```

Audit Log Table

```
-- Track all session mutations for fraud detection
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  table_name VARCHAR(50) NOT NULL,
  operation VARCHAR(10) NOT NULL, -- INSERT, UPDATE, DELETE
  record_id UUID NOT NULL,
  old_values JSONB,
  new_values JSONB,
  changed_by UUID REFERENCES auth.users(id),
  changed_at TIMESTAMP DEFAULT NOW(),
  branch_id UUID NOT NULL
);

-- Enable RLS on audit_log
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;

-- Append-only policy: admins can read, only trigger can insert
CREATE POLICY audit_log_insert_only ON audit_log
FOR INSERT WITH CHECK (true);

CREATE POLICY audit_log_select_admin ON audit_log
FOR SELECT USING (
  EXISTS (
    SELECT 1 FROM profiles
    WHERE id = auth.uid()
    AND role = 'admin'
  )
);

-- Trigger to log all session changes
CREATE OR REPLACE FUNCTION log_session_changes()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO audit_log (table_name, operation, record_id, old_values,
new_values, changed_by, branch_id)
  VALUES (
    'sessions',
    TG_OP,
    COALESCE(NEW.id, OLD.id),
    to_jsonb(OLD),
    to_jsonb(NEW),
    auth.uid(),
    COALESCE(NEW.branch_id, OLD.branch_id)
  );
END;
```

```

);
RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER sessions_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON sessions
FOR EACH ROW EXECUTE FUNCTION log_session_changes();

```

5 اختبارات الوحدة

الملف: `src/lib/__tests__/pricing.test.ts`

```

describe('calculateSessionPrice', () => {
  it('charges a minimum of 1 minute even for very short sessions', () => {
    // 10 seconds at 60 EGP/hr → should charge for 1 minute = 1 EGP
    const price = calculateSessionPrice(10, 60);
    expect(price).toBeCloseTo(1, 2);
  });

  it('rounds UP partial minutes (ceil, not floor)', () => {
    // 1 min 30 sec → 2 minutes (ceil)
    const price = calculateSessionPrice(90, 60);
    expect(price).toBeCloseTo(2, 2);
  });

  it('never returns a negative price', () => {
    const price = calculateSessionPrice(-500, 60);
    expect(price).toBeGreaterThanOrEqual(0);
  });

  it('handles zero hourly rate correctly', () => {
    const price = calculateSessionPrice(3600, 0);
    expect(price).toBe(0);
  });
});

```

تشغيل الاختبارات:

```
pnpm test
# ✔ 27 tests PASSED
```

6 Supabase تعليمات النشر على

الخطوة 1: تطبيق هجرة قاعدة البيانات

```
# 1. Open Supabase SQL Editor
# 2. Copy the entire content of:
#   drizzle/migrations/001_security_hardening.sql
# 3. Paste into SQL Editor and execute
```

RLS الخطوة 2: التحقق من

```
-- Verify RLS is enabled on audit_log
SELECT tablename, rowsecurity
FROM pg_tables
WHERE tablename = 'audit_log';
-- Should return: true

-- Verify policies exist
SELECT policyname, qual
FROM pg_policies
WHERE tablename = 'audit_log';
```

RPC الخطوة 3: اختبار دوال

```
-- Test start_session function
SELECT * FROM start_session(
  p_device_id := 1,
  p_customer_id := NULL,
  p_mode := 'single',
  p_hourly_rate := 50
);

-- Test stop_session function
SELECT * FROM stop_session(
  p_session_id := 'YOUR_SESSION_ID'
);
```

Ghost Session Reaper الخطوة 4: تفعيل

```
-- Enable pg_cron extension
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Schedule the ghost session closer
SELECT cron.schedule('close-ghost-sessions', '0 * * * *',
  'SELECT close_ghost_sessions()');

-- Verify schedule
SELECT * FROM cron.job;
```

7 ملخص الملفات المعدلة

التغييرات	الوصف	الملف
رسائل عربية، تصنيف الأخطاء	معالجة الأخطاء	src/lib/errors.ts
بدلاً من التحديثات RPC المباشرة	عمليات الجلسة	src/lib/sessions.ts
RBAC مساعدات	التحكم في الأدوار	src/lib/auth-context.tsx
branch_id، isGhostRisk تصفية	إدارة الأجهزة	src/hooks/useDevices.ts
حماية من النقرات، تحذيرات	بطاقة الجهاز	src/components/devices/DeviceCard.tsx
الفهارس، الدوال، التدقيق	هجرة قاعدة البيانات	drizzle/migrations/001_security_hardening.sql
اختبار 27	اختبارات التسعير	src/lib/__tests__/pricing.test.ts

8 الميزات الأمنية الموجودة

- ✓ منع تكرار الجلسات - فهرس فريد + قفل على مستوى الصف
- ✓ حماية من التلاعب بالأسعار - حساب على الخادم فقط
- ✓ Realtime في branch_id عزل البيانات متعدد المستأجرين - تصفية
- ✓ اكتشاف الجلسات المهجورة - تحذير بعد 6 ساعات، إغلاق تلقائي بعد 12 ساعة
- ✓ isProcessing حماية من النقرات المتكررة - حالة
- ✓ معالجة الأخطاء الآمنة - رسائل عربية، بدون تفاصيل حساسة
- ✓ مع المشغلات audit_log تدقيق كامل - جدول
- ✓ للأسعار والمدد CHECK constraints - قيود مالية
- ✓ اختبارات شاملة - 27 اختبار للتسعير

9 الخطوات التالية

1. Supabase تطبيق الهجرة على - `security_hardenning.sql` انسخ محتوى 001
 2. `start_session` g `stop_session` تحقق من - RPC اختبار دوال
 3. `pnpm test` - تشغيل الاختبارات (27 تمر 27)
 4. Manus Publish button نشر على الإنتاج - استخدم
 5. `audit_log` مراقبة - تحقق من السجلات للأنشطة المرئية
-

تاريخ الإنشاء: 2 أبريل 2026

الإصدار: 1.0.0

الحالة:  جاهز للنشر